



Advanced High Performance Computing Exercise Sheet 9

Dr. Douglas Potter

<http://www.ics.uzh.ch/>

Teaching Assistants:

André Glatzl, andre.glatzl@uzh.ch

Mark Eberlein, mark.eberlein@uzh.ch

Issued: 24.04.2024

Due: 08.05.2024

Exercise 1 [1 point] Ghost region

The `local_n0` variable contains the number of slabs that are needed for the current rank. Create a new variable `slab_count` where you add additional ghost slabs based on the mass assignment order. Nearest grid point (NGP) does not need a ghost region, while higher order schemes need an increasing number of ghost slabs.

From this you need to calculate how many floats you need to allocate. FFTW has helpfully given you how many complex numbers (pairs of floats) you need to allocate, and this may include additional memory for internal operations. The ghost region complicates things. You will need to calculate the number of floats you need based on `slab_count` and `nGrid`. Remember that the slab includes the padding that you had to deal with in an earlier exercise. You will need to allocate either the number of floats that FFTW has requested, or the number you need to handle the addition of the ghost region – whichever is **larger**.

Adjust the memory allocation for the slabs to use this new value. Create a new array `ghost` that is similar to `slab`, but contains the ghost cells. This means that you will adjust the number of slabs (first dimension) to include any ghost slabs. Should you do this for `kslab` (the complex view after the FFT)? HINT: Do you need/want a ghost region after the FFT?

Exercise 2 [1 point] Remove the large grid

There is no going back!

- Remove the code to create `grid` and `kgrid`
- Do the mass assignment to `ghost` instead of `grid`
- remove the code that combines the grids onto the distributed slabs

Careful: How should you “wrap” the slab index into `ghost` that you calculate? You do not

want negative indexes, but the ghost region can extend beyond the grid.

Exercise 3 [2 points] Merge Ghost regions onto the correct slab

You now need to “reduce” the ghost regions onto the correct slabs which are on another node. To simplify this we will make the following assumptions which are **not generally true**, but are usually true and will allow us to make the most simple change possible.

- The ghost region belongs to the next rank (the ghost region on rank 0 belongs to rank 1, 1 to 2, etc.)
- Exception: the ghost region on the last rank belongs to rank 0
- All ghost slabs on a rank belong to the **same** rank (i.e., the next rank)
- There are an even number of ranks

This is not generally true because:

- Some ranks may have no slabs
- Different ghost slabs on a rank may belong to different ranks
- The owning rank is decided by FFTW and may not be the “next” rank (but it is)

What we need to do is “reduce” the overlap regions (ghost slabs) in a ring fashion. Rank 0 needs to accumulate the “ghost region” onto rank 1, rank 1 onto rank 2, ... and so on until the last rank accumulates onto rank 0. We can’t call `MPI_Reduce` directly because then all ranks would participate, and only one would receive the results.

As we discussed, the solution is the “split” the MPI communicator `MPI_COMM_WORLD` with the `MPI_Comm_split` function. This call takes a `colour` and `key` parameter. Ranks with the same `colour` will be in the same communicator and they will be assigned a new rank starting from zero and by ordered by `key`.

When the communicator is split, it will make it easiest for the reduction operators if the same rank is always the one that receives the reduction (the **root shown in bold** in the tables below). Let’s take an example of 6 ranks. Each rank will need two new communicators, one as the receiver, and one as the sender. Rank 4 needs to send the ghost region to rank 5, but it also needs to receive the ghost region from rank 3.

This needs to be done in (at least) two steps. Call `MPI_Comm_split` to create three communicators with (0,1), (2,3), (4,5) and then call it again to create communicators (1,2), (3,4), (5,0). Read the documentation for `MPI_Comm_split`. The `key` parameter only determines the order, and does not even need to be unique. The rank inside the new communicator is determined

by ordering the old ranks by key and by old rank if the keys are the same. You can adopt any system you want, but for the first split you might try the following.

rank	colour	key	new rank
0	0	0	0
1	0	0	1
2	1	0	0
3	1	0	1
4	2	0	0
5	2	0	1

The trick is to come up with an expression for `colour` and `key` given `rank` to match the above table which gives us our first set of communicators. This might look something like:

```

1 MPI_Comm newcomm1;
2 int colour = rank / 2;
3 int key = 0;
4 MPI_Comm_split(MPI_COMM_WORLD, colour, key, &newcomm1);

```

For the second split, it is a little more complicated because it wraps, but here is a good starting point.

rank	colour	key	new rank
5	0	5	0
0	0	6	1
1	1	1	0
2	1	2	1
3	2	3	0
4	2	4	1

You can work out how to calculate `colour` and `key` to match the above table. For this to be truly general, you still have to handle the case where the number of ranks is odd. This involves more complexity and a third call to `MPI_Comm_split` where you sometimes set `colour` to `MPI_UNDEFINED`. For this exercise you can ignore this complication.

Now that each rank has its “send” and “receive” communicators you can call the `MPI_Reduce` function to do the pairwise reduction, but again there is a snag. You need to initiate the classic send/receive pair which would cause a deadlock. You have two choices. One solution is to issue the calls to `MPI_Reduce` in the correct order so that they match, so half start the “receive” part while the other matching ranks start the “send”, then they reverse rolls (and change partners). This could mean that even ranks would send first, then receive while odd ranks would receive then send. Another way is to issue both as non-blocking collectives

by calling `MPI_Ireduce` (twice) and then calling `MPI_Waitall` for them to complete.

Exercise 4 [1 point] Get the correct result

You used `blitz::sum` on the local slabs to get a partial sum of the mass, then called `MPI_Allreduce` to get the global mass so that you could calculate delta. You still need to do this, but should the partial sums be done on the `ghost` grid? You do not want to count mass twice so you have two choices.

- (i) Calculate the partial sum on `ghost` before you reduce the slabs (previous exercise)
- (ii) Calculate the partial sum on `grid` after

Verify that you get the same measurement as before.

Exercise 5 [BONUS 2 points] Odd number of ranks

Make your code work for an odd number of ranks (including 1).